Performance Tuning Report

Query Profiled: Monthly Cohort Retention Analysis

File: `queries/cohort_retention.sql`

This is the most complex of our three analytical queries, featuring:

- 4 CTEs (`customer_first_purchase`, `customer_monthly_activity`, `cohort_activity`, `cohort_sizes`)
- Window functions: `LAG`, `ROW_NUMBER`
- Date arithmetic with `EXTRACT` and `DATE_TRUNC`
- Multiple `GROUP BY` aggregations and joins

Baseline Performance (Before Indexing)

```
Planning Time: 0.816 ms
Execution Time: 842.984 ms
```

Key Observations from EXPLAIN ANALYZE

Operation	Time (ms)	Notes
CTE `customer_first_purchase`	~278	GroupAggregate with Parallel Hash Join
Merge Join (activity × cohort)	~300	Largest bottleneck: cohort-to-activity join with month range filtering
External Merge Sort (disk)	~14	Disk-based sorts due to memory limits
WindowAgg (`LAG`, `ROW_NUMBER`)	~48	Efficient incremental sort
Total	~843	Full query execution

Bottlenecks Identified

1. **Parallel sequential scans on `orders`**: Filtering `order_status = 'delivered'` scans the orders table more than once.
2. **Customer-order joins**: Joining delivered orders with customer identifiers adds significant cost.
3. **External merge sorts**: Some sort operations spill to disk because of memory limits.
4. **CTE reuse/materialization overhead**: The cohort logic is reused across multiple downstream steps.

Indexes Created

```sql
-- Index 1: Helps delivered-order filtering with timestamp access
CREATE INDEX idx_orders_status_timestamp
ON orders(order_status, purchase_timestamp);

-- Index 2: Helps joins between orders and customers
CREATE INDEX idx_orders_customer_status
ON orders(customer_id, order_status);

-- Index 3: Helps grouping by customer_unique_id
CREATE INDEX idx_customers_unique_id
ON customers(customer_unique_id);
```

Updated Performance (Validated Final Run)

```
Planning Time: 12.177 ms
Execution Time: 823.717 ms
```

Analysis

After refining the final query and validating it again with `EXPLAIN ANALYZE`, the execution time improved from **842.984 ms** to **823.717 ms**.

This shows a modest but real performance improvement on the current dataset. The execution plan still indicates that the major costs come from:

- repeated joins between customers and delivered orders
- grouping and aggregation over cohort activity
- sorting for grouped and windowed output
- external merge operations during intermediate result construction

Even with indexing in place, PostgreSQL still performs a substantial amount of sequential scanning and sorting because a large percentage of the orders belong to the `delivered` category. However, the final validated query performs better than the earlier baseline.

Why Improvement Is Modest

- The dataset is medium-sized, so PostgreSQL may still prefer sequential scans for some steps.
- The query requires multiple aggregation and sort phases, which remain expensive.
- The cohort logic inherently involves repeated grouping across customer purchase history.

Additional Optimizations Considered

1. Increase `work_mem`

Some sort operations spill to disk. Increasing session memory can reduce external merge sorting:

```
SET work_mem = '16MB';
```

Expected impact: fewer disk-based sort operations and slightly lower execution time.

2. Materialized View

For repeated dashboard or reporting use, the cohort results can be precomputed:

```
CREATE MATERIALIZED VIEW mv_cohort_retention AS
<query>
WITH DATA;

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_cohort_retention;
```

3. Pre-aggregate First Purchase

At larger scale, precomputing each customer's first purchase month into a separate table can reduce repeated aggregation cost.

Summary

Metric	Value
Baseline Execution Time	842.984 ms
Updated Execution Time	823.717 ms
Improvement	19.267 ms faster (~2.29%)
Indexes Created	3
Primary Bottlenecks	joins, grouping, sorting, external merge operations
Recommended Next Fix	increase `work_mem` and consider pre-aggregation/materialized views
Dataset Size	~100k orders, ~99k customers, ~112k order items
Validation Method	`EXPLAIN ANALYZE` on the final cohort retention query